3203 OPERATING SYSTEM • SILBERSCHATZ, GALVIN & GAGNE

# Chapter 6: Process Synchronization & Synchronization Tools

Complete Study Guide: Critical-Section Problem, Hardware Primitives, Semaphores, Monitors, Classical Problems & Liveness

### CHAPTER OBJECTIVES & SYLLABUS SCOPE

- **Concept of Process Synchronization:** Understand how concurrent or parallel execution affects the integrity of shared data in a shared-memory environment and how race conditions arise.
- **The Critical-Section Problem:** Formulate protocols that ensure data consistency via Mutual Exclusion, Progress, and Bounded Waiting across cooperating processes.
- **Software & Hardware Solutions:** Analyze strict alternation, Peterson's Solution, memory models and memory barriers, hardware atomic instructions ( `test_and_set()` and `compare_and_swap()` ), and atomic variables.
- **Synchronization Tools:** Master mutex locks, spinlocks, binary and counting semaphores, monitors, condition variables, and semaphore-based monitor implementations.
- **Classical Problems, Liveness & Evaluation:** Examine the Bounded-Buffer, Readers–Writers, and Dining-Philosophers problems, analyze liveness hazards (deadlock, starvation, priority inversion), and evaluate contention loads.

## 6.1 BACKGROUND & RACE CONDITIONS

A **cooperating process** is one that can affect or be affected by other processes executing in the system. Cooperating processes can either directly share a logical address space (both code and data, as in multithreaded programs) or share data through shared memory or message passing. Because the CPU scheduler switches rapidly between processes to provide *concurrent execution*, a process may be interrupted at any point in its instruction stream, partially completing execution before another process is scheduled. Similarly, under *parallel execution*, two instruction streams execute simultaneously on separate processing cores.

Concurrent or parallel access to shared data may result in data inconsistency. A **race condition** occurs when several processes access and manipulate the same shared data concurrently and the outcome of the execution depends on the particular order in which the accesses take place. For example, in the Bounded-Buffer Producer–Consumer problem, if both the producer (`counter++`) and consumer (`counter--`) manipulate a shared `counter` variable simultaneously, interleaved machine register loads and stores can leave `counter` with an incorrect value. Similarly, in an operating system kernel, if two processes concurrently call `fork()` and request the next available process identifier (`next_available_pid = 2615`), a race condition on `next_available_pid` causes both child processes to be assigned the identical PID (`2615`). Guarding against race conditions requires and coordination.**process synchronization**

## 6.2 THE CRITICAL-SECTION PROBLEM

Consider a system consisting of $n$ processes $\{P_0, P_1, \ldots, P_{n-1}\}$. Each process has a segment of code, called a **critical section**, in which the process may be changing common variables, updating a table, or writing a file. The fundamental invariant of the system is that when one process is executing in its critical section, no other process is allowed to execute in its critical section. The **critical-section problem** is to design a protocol that processes can use to cooperate safely.

- **Entry Section:** The code segment where a process requests permission to enter its critical section.
- **Critical Section:** The protected code segment accessing or modifying shared data.
- **Exit Section:** The code segment following the critical section that releases access permission.
- **Remainder Section:** All remaining code executed by the process outside the critical protocol.

> **Figure 6.1: General Structure of a Typical Process** `P_i do { /* [ ENTRY SECTION ] - Ask permission to enter critical section */ critical section /* [ EXIT SECTION ] - Release permission after exiting critical section */ remainder section } while (true);`

### Three Mandatory Requirements for a Valid Solution

Assuming each process executes at a nonzero speed—and making **no assumption concerning the relative speed** of the $n$ processes—any valid solution to the critical-section problem must satisfy three requirements:

| Requirement | Formal Textbook Definition | System Guarantee |
|---|---|---|
| **1. Mutual Exclusion** | If process $P_i$ is executing in its critical section, then no other processes can be executing in their critical sections. | Safety: Only one process is active in its critical section at a time. |

| Requirement | Formal Textbook Definition | System Guarantee |
|---|---|---|
| **2. Progress** | If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely. | Liveness: Prevents deadlock and blocks remainder-section processes from obstructing entry. |
| **3. Bounded Waiting** | There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted. | Fairness: Limits how long a process waits before entering its critical section (no starvation). |

### Critical-Section Handling in Operating Systems

Kernel data structures (structures for maintaining memory allocation, process lists, open file tables, and interrupt handling) are subject to race conditions. Two approaches are used to handle critical sections in OS kernels depending on whether the kernel is preemptive or nonpreemptive:

- **Preemptive Kernel:** Allows a process to be preempted while it is running in kernel mode. Preemptive kernels must be carefully designed to be free from race conditions (especially on SMP multiprocessors), but are preferred because they are more responsive and better suited for real-time programming.
- **Nonpreemptive Kernel:** Does not allow a process running in kernel mode to be preempted; a kernel-mode process will run until it exits kernel mode, blocks, or voluntarily yields control of the CPU. This is essentially free from race conditions on kernel data structures because only one process is active in the kernel at a time.

## 6.3 PETERSON'S SOLUTION

A naive two-process algorithm that alternates a shared variable `turn` (`while (turn == j);` in the entry section and `turn = j;` in the exit section) satisfies mutual exclusion, but violates **progress** because it enforces strict alternation (if $P_0$ is in its remainder section and `turn == 0`, $P_1$ is blocked from entering). **Peterson's Solution** is a classic software-based solution restricted to two processes ($P_0$ and $P_1$) that assumes basic machine-language `load` and `store` instructions are atomic.

The two processes share two data items: `int turn;` and `boolean flag[2];`. The variable `turn` indicates whose turn it is to enter its critical section, while `flag[i] = true` indicates that process $P_i$ is ready to enter its critical section. To enter the critical section, process $P_i$ first sets `flag[i] = true` and then sets `turn = j`, graciously asserting that if the other process wishes to enter, it can do so. If both processes attempt to enter at the same time, `turn` will be overwritten with either $i$ or $j$, and the final value of `turn` decides which process enters first.

```
Peterson's Solution for Process P_i (Satisfies Mutual Exclusion, Progress, and Bounded Waiting) int turn; boolean flag[2]; do { flag[i] = true; /* P_i states intent to enter critical section */ turn = j; /* P_i gives priority to P_j in case of tie */ while (flag[j] && turn == j); /* Wait only if P_j also wants in AND it is P_j's turn */ /* critical section */ flag[i] = false; /* Exit section: P_i leaves critical section */ /* remainder section */ } while (true);
```

**Proof of Correctness & Instruction Reordering Limitation:** (1) *Mutual exclusion* is preserved because $P_i$ enters its critical section only if either `flag[j] == false` or `turn == i`; if both processes are in their critical sections, `flag[0] == flag[1] == true`, but `turn` cannot be both `0` and `1` simultaneously. (2) *Progress* and (3) *Bounded waiting* (at most 1 entry by $P_j$) hold because a blocked process in the `while` loop immediately proceeds once the other sets `flag[j] = false` or resets `turn`. However, Peterson's solution is **not guaranteed to work on modern computer architectures** because processors and compilers may reorder dependent-free read and write operations to improve performance. If $P_0$'s assignments in the entry section are reordered so that `turn = 1` executes before `flag[0] = true`, both processes can evaluate their `while` conditions as false at different moments and enter their critical sections concurrently.

## 6.4 HARDWARE SUPPORT FOR SYNCHRONIZATION

In a single-processor environment, the critical-section problem could be solved simply by **preventing interrupts** from occurring while a shared variable is being modified. Unfortunately, disabling interrupts is not feasible in a **multiprocessor environment**: passing interrupt-disable messages to all processors is time-consuming, delays entry into every critical section, decreases system efficiency, and can degrade a system clock updated by interrupts. Instead, modern computer architectures provide three hardware mechanisms: **memory barriers**, **hardware instructions**, and **atomic variables**.

### 1. Memory Models & Memory Barriers

A computer architecture's **memory model** specifies the memory guarantees made to application programs and falls into one of two categories: (1) **Strongly ordered**, where a memory modification of one processor is immediately visible to all other processors, or (2) **Weakly ordered**, where modifications to memory on one processor may not be immediately visible to other processors. To overcome instruction reordering and delayed visibility, hardware provides **memory barriers** (also known as *memory fences*)—instructions that force any change in memory to be propagated (made visible) to all other processors before any subsequent load or store operations are performed:

```
Enforcing Memory Visibility with memory_barrier() /* Thread 1: Waits on flag, then reads x */ /* Thread 2: Updates x, then signals flag */ while (!flag) x = 100; memory_barrier(); memory_barrier(); print x; flag = true;
```

### 2. Hardware Instructions: `test_and_set()` and `compare_and_swap()`

Many modern machines provide special hardware instructions that allow us either to test and modify the content of a word or to swap the contents of two words **atomically**—that is, as one uninterruptible unit. If two `test_and_set()` or `compare_and_swap()` instructions are executed simultaneously (each on a different core), they will be executed sequentially in some arbitrary order.

**1. The test_and_set() Instruction and Mutual-Exclusion Implementation**

```
boolean test_and_set(boolean *target) { boolean rv = *target; *target = true; return rv; } /* Shared boolean variable 'lock' initialized to false */ do { while (test_and_set(&lock)) ; /* do nothing (busy wait) */ /* critical section */ lock = false; /* remainder section */ } while (true);
```

**2. The compare_and_swap() (CAS) Instruction and Mutual-Exclusion Implementation**

```
int compare_and_swap(int *value, int expected, int new_value) { int temp = *value; if (*value == expected) *value = new_value; return temp; } /* Shared integer 'lock' initialized to 0 */ while (true) { while (compare_and_swap(&lock, 0, 1) != 0) ; /* do nothing (busy wait) */ /* critical section */ lock = 0; /* remainder section */ }
```

### Bounded-Waiting Mutual Exclusion & 3. Atomic Variables

While the basic `test_and_set()` and `compare_and_swap()` locks above satisfy Mutual Exclusion and Progress, they do **not** satisfy Bounded Waiting. Below is the complete algorithm that satisfies all three requirements using common data structures `boolean waiting[n];` (initialized to `false`) and `int lock;` (initialized to `0`). A process $P_i$ can enter its critical section only if either `waiting[i] == false` or `key == 0`. When exiting, $P_i$ scans the `waiting` array cyclically (`j = (i + 1) % n`) and designates the first waiting process as the next one to enter the critical section, guaranteeing any waiting process enters within at most $n - 1$ turns.

**Bounded-Waiting Mutual Exclusion with compare_and_swap() (or test_and_set())**

```
while (true) { waiting[i] = true; key = 1; while (waiting[i] && key == 1) key = compare_and_swap(&lock, 0, 1); waiting[i] = false; /* critical section */ j = (i + 1) % n; while ((j != i) && !waiting[j]) j = (j + 1) % n; if (j == i) lock = 0; else waiting[j] = false; /* remainder section */ }
```

**Atomic Variables:** Typically, the `compare_and_swap()` instruction is not used directly to provide mutual exclusion; rather, it is used as a basic building block for constructing higher-level tools such as **atomic variables**. An atomic variable provides atomic (uninterruptible) operations on basic data types such as integers and booleans. For example, an atomic increment function `increment(&sequence)` can be implemented safely without locks using a CAS loop:

**Atomic Increment Implemented via compare_and_swap()**

```
void increment(atomic_int *v) { int temp; do { temp = *v; } while (temp != compare_and_swap(v, temp, temp + 1)); }
```

## 6.5 MUTEX LOCKS & SPINLOCKS

Hardware-based solutions are complicated and generally inaccessible to application programmers. Instead, operating-system designers build software tools to solve the critical-section problem. The simplest of these tools is the **mutex lock** (short for *mutual exclusion*). We use the mutex lock to protect critical sections and thus prevent race conditions: a process must `acquire()` the lock before entering a critical section, and it `release()`s the lock when it exits the critical section. A mutex lock has a boolean variable `available` whose value indicates if the lock is available or not. Calls to `acquire()` and `release()` must be performed atomically (usually implemented via hardware atomic instructions such as `compare_and_swap()`).

**Mutex Lock Operations: acquire() and release()**

```
acquire() { while (!available) ; /* busy wait */ available = false; } release() { available = true; }
```

**Busy Waiting and Spinlocks:** The main disadvantage of the implementation given above is that it requires **busy waiting**. While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the call to `acquire()`. This type of mutex lock is called a **spinlock** because the process "spins" while waiting for the lock to become available. Continual looping is clearly a problem in a real multiprogramming system, where a single CPU core is shared among many processes, because busy waiting wastes CPU cycles that some other process might be able to use productively. However, spinlocks do have an important advantage on **multiprocessor systems**: *no context switch is required* when a process must wait on a lock, and a context switch may take considerable time. When locks are expected to be held for short times (generally less than the time required for two context switches), one thread can "spin" on one processing core while another thread performs its critical section on another core.

## 6.6 SEMAPHORES

A **semaphore** *S* is an integer variable that, apart from initialization, is accessed only through two standard atomic operations: `wait()` and `signal()`. The `wait()` operation was originally termed **P** (from the Dutch *proberen*, "to test"); `signal()` was originally called **V** (from *verhogen*, "to increment"). All modifications to the integer value of the semaphore in the `wait()` and `signal()` operations must be executed atomically.

```
Standard Definition of Semaphore wait() and signal() wait(S) { while (S <= 0) ; /* busy wait */ S--; } signal(S) { S++; }
```

### Semaphore Usage: Counting vs. Binary Semaphores

- **Counting Semaphore:** The value of a counting semaphore can range over an unrestricted domain. Counting semaphores can be used to control access to a given resource consisting of a finite number of instances. The semaphore is initialized to the number of resources available. Each process that wishes to use a resource performs a `wait()` operation on the semaphore (thereby decrementing the count). When a process releases a resource, it performs a `signal()` operation (incrementing the count). When the count for the semaphore goes to `0`, all resources are being used, and processes that wish to use a resource will block until the count becomes greater than `0`.
- **Binary Semaphore:** The value of a binary semaphore can range only between `0` and `1`. Thus, binary semaphores behave similarly to mutex locks and can be used on systems that do not provide mutex locks directly.
- **Execution Ordering Synchronization:** Semaphores can also solve various synchronization problems. For example, consider two concurrently running processes: $P_1$ with a statement $S_1$ and $P_2$ with a statement $S_2$. Suppose we require that $S_2$ be executed only after $S_1$ has completed. We can implement this scheme readily by letting $P_1$ and $P_2$ share a common semaphore `synch`, initialized to `0`. In process $P_1$, we insert the statements `S1; signal(synch);` and in process $P_2$, we insert the statements `wait(synch); S2;`. Because `synch` is initialized to `0`, $P_2$ will block and execute $S_2$ only after $P_1$ has invoked `signal(synch)`.

### Semaphore Implementation with no Busy Waiting

To overcome the need for busy waiting in the `wait()` operation, we can modify the definition of the `wait()` and `signal()` operations. When a process executes the `wait()` operation and finds that the semaphore value is not positive, it must wait. However, rather than engaging in busy waiting, the process can **block itself** via the `sleep()` operation, which places the process into a waiting queue associated with the semaphore, and the state of the process is switched to the waiting state. Then control is transferred to the CPU scheduler, which selects another process to execute. A process that is blocked waiting on a semaphore $S$ should be restarted when some other process executes a `signal()` operation via a `wakeup(P)` operation, which changes the process from the waiting state to the ready state and places it in the ready queue. Notice that in this implementation, semaphore values may be negative, and the magnitude of a negative value equals the number of processes waiting on that semaphore.

**Semaphore Implementation without Busy Waiting (Using Waiting Queues)**

```
typedef struct { int value; struct process *list; } semaphore;
wait(semaphore *S) { S->value--; if (S->value < 0) { add this process to S->list; sleep(); } }
signal(semaphore *S) { S->value++; if (S->value <= 0) { remove a process P from S->list; wakeup(P); } }
```

## 6.7 MONITORS & CONDITION VARIABLES

Although semaphores provide a convenient and effective mechanism for process synchronization, using them incorrectly can result in timing errors that are difficult to detect, since these errors happen only if particular execution sequences take place:

1. **Interchanging `wait()` and `signal()`:** Suppose a process interchanges the order in which the `wait()` and `signal()` operations on the semaphore `mutex` are executed, resulting in `signal(mutex); ... critical section ... wait(mutex);`. In this situation, several processes may be executing in their critical sections simultaneously, violating the mutual-exclusion requirement.
2. **Replacing `signal()` with `wait()`:** Suppose a program replaces `signal(mutex)` with `wait(mutex)`: `wait(mutex); ... critical section ... wait(mutex);`. In this case, a deadlock will occur (the process permanently blocks on the second call to `wait()`).
3. **Omitting `wait()` or `signal()`:** Suppose a process omits the `wait(mutex)`, or the `signal(mutex)`, or both. In this case, either mutual exclusion is violated or a deadlock will occur.

### Monitor Usage & Condition Variables

To deal with such errors, researchers developed a fundamental high-level synchronization construct—the **monitor** type. A monitor type is an abstract data type (ADT) that includes a set of programmer-defined operations that are provided with mutual exclusion within the monitor. The monitor type also declares the variables whose values define the state of an instance of that type, along with the bodies of functions that operate on those variables. A procedure defined within a monitor can access only those variables declared locally within the monitor and its formal parameters. Similarly, the local variables of a monitor can be accessed by only the local procedures. Most importantly, **the monitor construct ensures that only one process at a time is active within the monitor**.

However, the monitor construct, as defined so far, is not sufficiently powerful for modeling some synchronization schemes. To tailor a monitor to specific synchronization schemes, we need to define additional synchronization mechanisms provided by the **condition** construct: `condition x, y;`. The only operations that can be invoked on a condition variable are `wait()` and `signal()`:

- `x.wait();` means that the process invoking this operation is suspended until another process invokes `x.signal();`.
- `x.signal();` resumes **exactly one** suspended process. If no process is suspended, then the `signal()` operation has no effect; that is, the state of `x` is the same as if the operation had never been executed. (Contrast this with the `signal()` operation associated with semaphores, which always affects the state of the semaphore.)

Suppose that, when the `x.signal()` operation is invoked by a process *P*, there exists a suspended process *Q* associated with condition `x`. Clearly, if the suspended process *Q* is allowed to resume its execution, the signaling process *P* must wait. Otherwise, both *P* and *Q* would be active simultaneously within the monitor. Two possibilities exist: (1) **Signal and wait**—*P* either waits until *Q* leaves the monitor or waits for another condition; (2) **Signal and continue**—*Q* either waits until *P* leaves the monitor or waits for another condition.

```
Pseudocode Syntax of a Monitor Construct monitor monitor_name { /* shared variable declarations */ condition x, y; function P1 (...) { ... } function P2 (...) { ... } function Pn (...) { ... } initialization_code (...) { ... } }
```

### Implementing a Monitor Using Semaphores (Section 6.7.2)

We can implement the monitor mechanism using semaphores under the *Signal and wait* discipline. For each monitor, a binary semaphore `mutex` (initialized to `1`) is provided to ensure mutual exclusion. Because a signaling process must wait until the resumed process either leaves or waits, an additional binary semaphore, `next`, is introduced, initialized to `0`, along with an integer variable `next_count` initialized to `0` to count the number of processes suspended on `next`. For each condition `x`, we introduce a binary semaphore `x_sem` and an integer variable `x_count`, both initialized to `0`:

```
Implementing Monitor Procedures and Condition Variables via Semaphores /* 1. Wrapping Each External Function F in the Monitor */ wait(mutex); ... body of F ... if (next_count > 0) signal(next); else signal(mutex); /* 2. Implementation of Condition x.wait() */ /* 3. Implementation of Condition x.signal() */ x_count++; if (x_count > 0) { if (next_count > 0) next_count++; signal(next); signal(x_sem); else wait(next); signal(mutex); next_count--; wait(x_sem); } x_count--;
```

**Resuming Processes within a Monitor (Conditional-Wait):** If several processes are suspended on condition , and an operation is executed, how do we determine which suspended process should be resumed next? One simple scheme is to use a First-Come, First-Served (FCFS) ordering. When FCFS is not adequate, the construct can be used: , where is an integer expression evaluated when the operation is executed. The value of , which is called a , is then stored with the name of the process that is suspended. When is executed, the process with the smallest priority number is resumed next (as illustrated in a monitor that allocates a single resource among competing processes based on shortest maximum allocation duration). `x x.signal()` **conditional-wait** `x.wait(c); c wait() c` **priority number** `x.signal() ResourceAllocator`

## CLASSICAL PROBLEMS OF SYNCHRONIZATION (COURSE LECTURE SLIDES & CH. 7.1)

In the course lecture slides and textbook, synchronization tools are applied to three classical synchronization problems used to test nearly every newly proposed synchronization scheme:

| Classical Problem | Shared Data & Semaphores | Core Protocol & Deadlock / Starvation Notes |
|---|---|---|
| **1. Bounded-Buffer Problem** | `n` buffers; `semaphore mutex = 1` (mutual exclusion); `semaphore empty = n`; `semaphore full = 0`. | **Producer:** `wait(empty); wait(mutex);` [add item] `signal(mutex); signal(full);` **Consumer:** `wait(full); wait(mutex);` [remove item] `signal(mutex); signal(empty);` |
| **2. Readers–Writers Problem** | Shared dataset; `semaphore rw_mutex = 1` (shared by readers and writers); `semaphore mutex = 1` (protects `read_count`); `int read_count = 0`. | **First Variations:** (1) No reader kept waiting unless a writer holds permission (may starve writers); (2) Once a writer is ready, it performs its write as soon as possible (may starve readers). First reader locks `wait(rw_mutex)`; last reader releases `signal(rw_mutex)`. |
| **3. Dining-Philosophers Problem** | 5 philosophers, 5 chopsticks: `semaphore chopstick[5]` initialized to `1` (or Monitor with states `THINKING`, `HUNGRY`, `EATING` and `condition self[5]`). | Naive `wait(chopstick[i]); wait(chopstick[(i+1)%5]);` causes **deadlock** if all 5 grab their left chopstick at once. **Deadlock handling remedies:** (1) Allow at most 4 philosophers at table; (2) Pick up chopsticks only if both are available in a critical section; (3) Asymmetric solution (odd picks left then right; even picks right then left). |

```
Monitor Solution to the Dining-Philosophers Problem (Deadlock-Free) monitor DiningPhilosophers { enum {THINKING, HUNGRY, EATING} state[5]; condition self[5]; void pickup(int i) { state[i] = HUNGRY; test(i); if (state[i] != EATING) self[i].wait(); } void putdown(int i) { state[i] = THINKING; test((i + 4) % 5); /* test left and right neighbors */ test((i + 1) % 5); } void test(int i) { if ((state[(i + 4) % 5] != EATING) && (state[i] == HUNGRY) && (state[(i + 1) % 5] != EATING)) { state[i] = EATING; self[i].signal(); } } initialization_code() { for (int i = 0; i < 5; i++) state[i] = THINKING; } }
```

## 6.8 LIVENESS & 6.9 EVALUATION OF SYNCHRONIZATION TOOLS

**Liveness** refers to a set of properties that a system must satisfy to ensure that processes make progress during their execution life cycle. A process waiting indefinitely is an example of a "liveness failure":

- **Deadlock:** Two or more processes are waiting indefinitely for an event that can be caused only by one of the waiting processes. For example, consider two semaphores $S$ and $Q$ initialized to `1`: if $P_0$ executes `wait(S); wait(Q);` and $P_1$ concurrently executes `wait(Q); wait(S);`, once $P_0$ acquires $S$ and $P_1$ acquires $Q$, neither can proceed.
- **Indefinite Blocking (Starvation):** A situation in which processes wait indefinitely within the semaphore. Indefinite blocking may occur if we remove processes from the list associated with a semaphore in Last-In, First-Out (LIFO) order rather than FIFO.
- **Priority Inversion:** A scheduling challenge that arises when a higher-priority process needs to read or modify kernel data that are currently being accessed by a lower-priority process—or a chain of lower-priority processes. Suppose we have three processes—$L$, $M$, and $H$—whose priorities follow the order $L < M < H$. Assume that process $H$ requires a semaphore $S$, which is currently being accessed by process $L$. Ordinarily, process $H$ would wait for $L$ to finish using resource $S$. However, now suppose that process $M$ becomes runnable, thereby preempting process $L$. Indirectly, a process with a lower priority—process $M$—has affected how long process $H$ must wait for $L$ to relinquish resource $S$. Typically, priority inversion is avoided by implementing a **priority-inheritance protocol**: all processes that are accessing resources needed by a higher-priority process inherit the higher priority until they are finished with the resources in question. When they are finished, their priorities revert to their original values.

### 6.9 Evaluation of Synchronization Tools Under Contention

When selecting among optimistic lock-free CAS-based synchronization and pessimistic traditional locking (mutexes, semaphores, monitors), performance varies substantially according to the level of **contention**:

| Contention Regime | CAS-Based (Lock-Free / Optimistic) Protection | Traditional Mutex / Semaphore Locks (Pessimistic) |
|---|---|---|
| **Uncontended / Low Contention** | **Fastest:** Single hardware instruction succeeds on first attempt with minimal overhead. | **Fast:** Slightly higher overhead than CAS, though lightweight user-space fast paths remain quick. |
| **Moderate Contention** | **Faster / Competitive:** CAS succeeds immediately or after very few retry loop iterations without context switching. | **Slower:** Failed lock acquisitions require expensive kernel wait-queue management and context switches. |
| **High Contention** | **Degrades Rapidly:** Repeated CAS failures create busy-wait retry storms and heavy bus/cache traffic. | **Superior:** Suspending contending threads in orderly kernel wait queues preserves CPU cycles for active work. |

### CHAPTER 6 PRACTICE EXERCISES — COMPLETE SOLUTIONS FOR EXAM PREPARATION

- **Ex 6.1 (Disabling Interrupts & System Clock):** Disabling interrupts frequently can affect the system's clock because the system clock is updated at every clock interrupt. If interrupts are disabled for a long period, the system clock can lose time. To minimize this effect, disable interrupts for only very short periods of time.
- **Ex 6.2 (Meaning of Busy Waiting):** Busy waiting means that a process is waiting for a condition to be satisfied in a tight loop without relinquishing the processor. Busy waiting can be avoided altogether by having the process block (`sleep()`) and placing it on a wait queue until another process wakes it up (`wakeup()`).
- **Ex 6.3 (Why Spinlocks Fail on Single-Core but Succeed on Multiprocessors):** On a single-processor system, the thread holding a lock cannot run while another thread is spinning on the CPU testing the lock; a preemption/context switch is guaranteed to be required anyway. On multiprocessor systems, one thread can spin on one core while the lock holder finishes its short critical section on another core, avoiding the cost of two context switches.
- **Ex 6.4 (Why wait() and signal() Must Be Atomic):** If `wait()` and `signal()` without busy waiting are not executed atomically, two processes might simultaneously decrement `S->value` or manipulate `S->list` concurrently, violating mutual exclusion or corrupting the semaphore wait queue.
- **Ex 6.5 (Binary Semaphore for n-Process Mutual Exclusion):** Initialize a shared binary semaphore `mutex = 1`. Each of the *n* processes executes `wait(mutex)` in its entry section before entering its critical section and `signal(mutex)` in its exit section upon leaving.
- **Ex 6.6 (Shared Bank Account Race Condition):** If a husband calls `withdraw(amount)` and a wife calls `deposit(amount)` concurrently on a shared `balance` variable, both functions may read the same initial `balance` into local registers, compute their respective results, and overwrite each other's updates. This race condition is prevented by treating the read-modify-write of `balance` as a critical section guarded by a mutex lock (`acquire()` / `release()`) or a binary semaphore.